

[login-א]

config/SecurityConfig.java

```
@Bean  
public PasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder();  
}
```

הגדרת BCrypt כמנגנון הצפנת סיסמאות. כל סיסמה שנשמרת עוברת תהליך hashing חד-כיווני – לא ניתן לשחזר ממנה את הסיסמה המקורית.

[login-ב]

auth/UsersPrePopulate.java

```
private void createUser(String username, UserAccount.Role role)  
{ String plainPwd =  
    java.util.UUID.randomUUID().toString().substring(0, 12); String  
    encoded = m_encoder.encode(plainPwd); m_users.add(new  
    UserAccount(username, encoded, role));  
    LOGGER.info("event=user_created username={}  
    tempPassword={}", username, plainPwd); }
```

אין סיסמאות קבועות בקוד. הסיסמה לכל משתמש נוצרת אקראית בזמן ריצה בעזרת UUID ומוצפנת מיידית ב BCrypt - הסיסמה הגולמית לא נשמרת בשום מקום - מודפסת רק ל log - לצורכי פיתוח.

```
2026-08-24T00:18:20.664+03:00 INFO 976 --- [BusyBee] [main] c.s.busybee.auth.UsersPrePopulate : event=user_created  
username=admin tempPassword=10be4e99-36b  
2026-08-24T00:18:20.742+03:00 INFO 976 --- [BusyBee] [main] c.s.busybee.auth.UsersPrePopulate : event=user_created  
username=alice tempPassword=d23058de-949  
2026-08-24T00:18:20.823+03:00 INFO 976 --- [BusyBee] [main] c.s.busybee.auth.UsersPrePopulate : event=user_created  
username=bob tempPassword=4e861ad2-274  
2026-08-24T00:18:20.895+03:00 INFO 976 --- [BusyBee] [main] c.s.busybee.auth.UsersPrePopulate : event=user_created  
username=charlie tempPassword=719f376f-1c2  
2026-08-24T00:18:20.896+03:00 INFO 976 --- [BusyBee] [main] c.s.busybee.auth.UsersPrePopulate : event=users_prepop  
ulated count=4
```

סעיפים מוסתרים:

[login-ג]

auth/UsernamePasswordDetailsService.java

```
LOGGER.warn("event=login_failed username={}", username);  
...  
LOGGER.info("event=login_success username={}", username);
```

לוגים של אירועי כניסה: כישלון והצלחה - מאפשרים זיהוי ניסיונות פריצה וחקירה לאחר מעשה.

[login-ד]

```
server.servlet.session.cookie.http-only=true  
server.servlet.session.cookie.secure=true  
server.servlet.session.cookie.same-site=Strict
```

Cookie של ה session-מוגן מ HttpOnly מונע גישה מ JavaScript, Secure מוודא שליחה רק על HTTPS, Strict = SameSite - מגן מפני CSRF.

[login-ה]

config/SecurityConfig.java

```
http.requiresChannel(channel ->  
    channel.anyRequest().requiresSecure());
```

הפניה כפויה מ-HTTP ל-HTTPS - כל בקשה שמגיעה על HTTP מנותבת אוטומטית לחיבור מאובטח.

[done-א]

controllers/DoneRequest.java

```
public record DoneRequest(@NotNull UUID taskid) {}
```

הארגומנט taskid מוגדר מסוג Java – UUID מוודאת אוטומטית שהערך הוא UUID חוקי. קלט שאינו UUID תקין ייכשל בפרסור עוד לפני הגעה ללוגיקה העסקית.

[done-ב]

```
@ExceptionHandler(TaskNotFoundException.class)  
public ResponseEntity<String>  
handleNotFound(TaskNotFoundException ex) {  
    return  
    ResponseEntity.status(HttpStatus.NOT_FOUND).body("Task not  
found");  
}
```

אם ה task-לא קיים, נזרקת חריגה TaskNotFoundException שנתפסת במנגנון המרכזי ומחזירה סטטוס 404 – ללא דליפת מידע על המבנה הפנימי.

[done-ג]

controllers/TasksController.java

```
@PostMapping("/done")

@PreAuthorize("@taskAuth.userAllowedToCloseTask(#request.taskId(), authentication)")

public ResponseEntity<String> markTaskDone(@RequestBody
DoneRequest request,

Authentication authentication)
```

auth/TaskAuthorization.java

```
public boolean userAllowedToCloseTask(UUID taskId, Authentication auth) {

    Task task =
m_tasks.findById(taskId).orElseThrow(TaskNotFoundException::new);

    String user = auth.getName();

    boolean isAdmin = auth.getAuthorities().stream()

        .anyMatch(a -> a.getAuthority().equals("ROLE_ADMIN"));

    return isAdmin || task.createdBy().equals(user);

}
```

הרשאות מוגדרות באמצעות SpEL ב – @PreAuthorize - רק בעל ה task-ו ADMIN יכולים לסגור אותו. כל שאר הבקשות מקבלות 403 אוטומטית.

[create-א]

```
public class TaskName extends BoundedWord {

    public TaskName(String value) throws TypeValidationException {

        super(value, 1, 50);

    }

}
```

TaskName מרחיב BoundedWord מספריית – OWASP SafeTypes מוודא שם של שורה בודדת, ללא תגיות, HTML באורך 1–50 תווים.

[create-ב]

```
private static final Safelist ALLOWED = Safelist.basicWithImages()

    .addTags("u")

    .preserveRelativeLinks(false);

...

String cleaned = Jsoup.clean(raw, ALLOWED);
```

ה desc-מנוקה עם Jsoup תוך שמירה על תגיות בטוחות בלבד <a>: קישורים , תמונות <u>, <i>, , עיצוב טקסט. כל תגית אחרת נמחקת. כך מתאפשרת הפונקציונליות הנדרשת.

[create-ג]

```
void blocksScriptTag() throws Exception {

    SafeDescription desc = new SafeDescription("<script>alert('xss')</script>Hello");
    assertFalse(desc.getValue().contains("<script>"));
}

void blocksJavascriptHref() throws Exception {

    SafeDescription desc = new SafeDescription("<a href='\"javascript:alert(1)\"'>click</a>");
    assertFalse(desc.getValue().contains("javascript:"));
}

void allowsSafeLink() throws Exception {

    SafeDescription desc = new SafeDescription("<a href='\"https://example.com\"'>link</a>");
    assertTrue(desc.getValue().contains("<a>"));
}
```

הבדיקות מוכיחות שמצד אחד XSS נחסם (script, javascript: href, onerror) ומצד שני הפונקציות הנדרשת נשמרת (קישורים, תמונות, עיצוב).

Test Summary

6	0	0	0.082s	100%
tests	failures	ignored	duration	successful
Packages	Classes			
Package	Tests	Failures	Ignored	Duration
com.securefromscratch.beebee.safety	6	0	0	0.082s

[create-ד]

```
@AssertTrue(message = "dueDate/dueTime must be in the future")
public boolean isDueDateTimeInFuture() {
    if (dueDate == null) return true;

    LocalDateTime due = LocalDateTime.of(dueDate,
        dueTime != null ? dueTime : LocalTime.MIDNIGHT);

    return due.isAfter(LocalDateTime.now());
}
```

dueDate וdueTime - מקבלים את הסוגים LocalDateTime וLocalTime - של Java - פרסור אוטומטי חוסם פורמטים לא חוקיים. בנוסף @AssertTrue, מוודא שהשילוב תאריך + שעה לא נמצא בעבר.

[create-ה]

```
public record CreateTaskRequest(TaskName name, SafeDescription desc,
    LocalDateTime dueDate, LocalTime dueTime, String[] responsibilityOf)
```

dueTime מסוג Java - LocalTime דוחה אוטומטית שעה לא חוקית (למשל 25:00) בפרסור. השילוב תאריך + שעה נבדק ב @AssertTrue - שמוודא שלא מדובר בזמן שכבר עבר.

[create-ר]

```
for (String username : request.responsibilityOf()) {  
    if (m_users.findByUsername(username).isEmpty()) {  
        return ResponseEntity.badRequest().body("Unknown user: " + username);  
    }  
}
```

כל שם בשדה responsibilityOf מול מאגר המשתמשים הקיים. שם שאינו מוכר מחזיר 400 Bad Request – מונע הוספת משתמשים פיקטיביים.

[create-ח]

```
if (m_tasks.findByName(request.name().toString()).isPresent()) {  
    return ResponseEntity.status(HttpStatus.CONFLICT).body("Task name already exists");  
}
```

אם כבר קיים task עם אותו שם, מוחזר סטטוס 409 Conflict – נמנעת יצירת tasks כפולים.

[create-ט]

```
@PostMapping("/create")  
public ResponseEntity<String> create(@Valid @RequestBody CreateTaskRequest request,  
    Authentication authentication) {  
    String createdBy = authentication.getName();  
    m_tasks.add(request.name().toString(), ..., createdBy);  
    ...  
}
```

הבעלים של task חדש נקבע אוטומטית לפי המשתמש המחובר מתוך – Authentication
לא ניתן לזייף בעלות על ידי שליחת ערך שונה ב request .

[create-י]

controllers/TasksController.java

```
@PostMapping("/create")  
@PreAuthorize("@taskAuth.isAuthenticated(authentication)")  
public ResponseEntity<String> create(...)
```

auth/TaskAuthorization.java

```
public boolean isAuthenticated(Authentication auth) {  
    boolean isAdmin = hasRole(auth, "ADMIN");  
    boolean isCreator = hasRole(auth, "CREATOR");  
    boolean isTrial = hasRole(auth, "TRIAL");  
    if (isAdmin || isCreator) return true;  
    if (isTrial) return m_tasks.findOpenTaskByUser(auth.getName()).isEmpty();  
    return false;  
}
```

ADMIN ו CREATOR יכולים ליצור tasks חופשי. TRIAL יכול ליצור רק אם אין לו task פתוח
– מונע שימוש לרעה בחשבונות ניסיון.

[create-יא]

controllers/TasksController.java

```
@PostMapping("/create")  
@PreAuthorize("@taskAuth.isAuthenticated(authentication)")  
public ResponseEntity<String> create(@Valid @RequestBody CreateTaskRequest  
request,  
Authentication authentication) {  
    String createdBy = authentication.getName();  
    m_tasks.add(request.name().toString(), ..., createdBy);  
    return ResponseEntity.ok("Task created");  
}
```

הקונטרולר אינו מכיל try-catch – הלוגיקה נקייה ומתמקדת בלבד בביצוע הפעולה.

controllers/GlobalExceptionHandler.java

```
@ControllerAdvice

public class GlobalExceptionHandler {

    @ExceptionHandler(TaskNotFoundException.class)

    public ResponseEntity<String> handleNotFound(TaskNotFoundException ex) {

        return ResponseEntity.status(HttpStatus.NOT_FOUND).body("Task not found");

    }

    @ExceptionHandler(MethodArgumentNotValidException.class)

    public ResponseEntity<String>
    handleValidation(MethodArgumentNotValidException ex) {

        return ResponseEntity.badRequest().body("Invalid input");

    }

    @ExceptionHandler(AccessDeniedException.class)

    public ResponseEntity<String> handleAccessDenied(AccessDeniedException ex)
    {

        return ResponseEntity.status(HttpStatus.FORBIDDEN).body("Access denied");

    }

}
```

כל החריגות מטופלות במנגנון מרכזי אחד (@ControllerAdvice) שממפה כל חריגה לסטטוס HTTP מתאים. כך נמנעת דליפת מידע פנימי, מובטחת עקביות בתגובות השגיאה, והקונטרולר נשמר נקי ללא בלוקי catch.

[comment-א]

storage/FileStorage.java

```
long usable = UPLOAD_DIR.toFile().getUsableSpace();

long size = file.getSize();

if (usable - size < MIN_FREE_BYTES) {

    LOGGER.warn("event=file_rejected reason=insufficient_disk_space
available={} required={}", usable, size);

    throw new IllegalArgumentException("Insufficient disk space");
}
```

בדיקת מקום פנוי בדיסק: לפני שמירת הקובץ, נבדק שיש מספיק מקום פנוי בדיסק. אם הקובץ גדול ממה שפנוי – מוחזר Bad Request 400 והקובץ לא נשמר כלל.

[comment-ב]

resources/application.properties

```
spring.servlet.multipart.max-file-size=5MB
```

controllers/CommentsUploadController.java

```
if (file.getSize() > MAX_FILE_SIZE) {

    throw new IllegalArgumentException("File exceeds maximum size of 5
MB");
}
```

הגבלת גודל קובץ: גודל מקסימלי מוגדר ברמת ה application-וגם נבדק בקוד - שכבה כפולה מונעת העלאת קבצים גדולים מדי.

[comment-ג]

storage/FileStorage.java

```
private static final Set<String> allowedExtensions =

    Set.of("jpg", "jpeg", "png", "gif", "webp", "pdf", "docx");

...

String extension = extension(originalName).toLowerCase();

if (!allowedExtensions.contains(extension)) {

    LOGGER.warn("event=file_rejected reason=extension_not_allowed ext={}",
extension);

    throw new IllegalArgumentException("File type not allowed: " + extension);
}
```

whitelist של סיומות מותרות: רק סיומות מוכרות ובטוחות מותרות – מונע העלאת קבצים הרצים (exe, sh, jsp וכו').

[comment-ד]

storage/FileStorage.java

```
private static final Map<String, byte[]> MAGIC_BYTES = Map.of(
    "jpg", new byte[]{(byte)0xFF, (byte)0xD8, (byte)0xFF},
    "png", new byte[]{(byte)0x89, 0x50, 0x4E, 0x47},
    "pdf", new byte[]{0x25, 0x50, 0x44, 0x46}
);
...
if (!startsWith(header, expected)) {
    LOGGER.warn("event=file_rejected reason=magic_bytes_mismatch ext={}", extension);
    throw new IllegalArgumentException("File content does not match declared extension");
}
```

אימות Magic Bytes : תוכן הקובץ נבדק לפי ה magic bytes-האמיתיים – מונע מתקיף לשנות סיומת לקובץ מורשה תוך שמירת תוכן זדוני.

[comment-ה]

storage/FileStorage.java

```
String storedFilename = UUID.randomUUID() + "_" + sanitize(originalName); Path
...
Files.createDirectories(UPLOAD_DIR);
Path target = UPLOAD_DIR.resolve(storedFilename).normalize();
```

שם קובץ אקראי (UUID) : שם הקובץ המקורי מוחלף ב UUID-אקראי לפני השמירה – מונע path traversal דרך שם הקובץ, התנגשויות בין קבצים, ודליפת מידע על מבנה הספרייה.

[comment-ו]

storage/FileStorage.java

```
Path target = UPLOAD_DIR.resolve(storedFilename).normalize();

if (!target.startsWith(UPLOAD_DIR)) {

    // Should be impossible given UUID prefix, but defense-in-depth

    throw new SecurityException("Path traversal detected");
}
```

Path Sandbox : אחרי בניית הנתיב מבצעים normalize() ובודקים שהתוצאה עדיין נמצאת בתוך תיקיית ה – upload-מונע path traversal גם אם UUID אקראי מכיל תווים חריגים.

[comment-ז]

storage/FileStorage.java

```
String browserMime = file.getContentType();

List<String> acceptedMimes = MIME_MAP.get(extension);

if (acceptedMimes != null && (browserMime == null ||
!acceptedMimes.contains(browserMime))) {

    LOGGER.warn("event=file_rejected reason=content_type_mismatch mime={}",
browserMime);
}
```

אימות Content-Type : Content-Type שנשלח ב request-נבדק מול רשימת סוגים מורשים – הגנה נוספת שמשלימה את בדיקת הסיומת ואת ה magic bytes.

[comment-ח]

controllers/CommentsUploadController.java

```
if (text == null || text.isBlank()) {

    throw new IllegalArgumentException("Comment text cannot be empty");

}

if (text.contains("<") || text.contains(">") || text.contains("&")) {

    throw new IllegalArgumentException("Comment text must be plain text, no
HTML allowed");

}

if (text.contains("\n") || text.contains("\r")) {

    throw new IllegalArgumentException("Comment text must be a single line");

}
```

ולידציה של טקסט התגובה: בודק שהטקסט בכלל קיים ולא ריק, בודק שאין תווים שיכולים להעיד על HTML, בודק שהתגובה היא שורה אחת בלבד.

[comment-ט]

storage/FileStorage.java

```
// Original filename is never stored or returned to the client  
String storedFilename = UUID.randomUUID() + "_" + sanitize(originalName);
```

סעיף מוסתר: אחסון בשם UUID בלבד: שם הקובץ המקורי אינו נשמר ואינו מוחזר ללקוח – מונע דליפת מידע ומניפולציה על שמות קבצים.

[image-א]

controllers/ImageController.java

```
private static final List<String>  
ALLOWED_EXTENSIONS =  
  
    List.of("jpg", "jpeg", "png", "gif", "webp");  
  
@GetMapping("/image")  
@PreAuthorize("@taskAuth.imgIsInOwnedOrAssignedTask(#img, authentication)")  
public ResponseEntity<byte[]>  
getImage(@RequestParam("img") ImageName img,  
...) {  
  
    // 1. Extension whitelist  
  
    String ext = extension(img.getName());  
  
    if (!ALLOWED_EXTENSIONS.contains(ext)) {  
  
        return ResponseEntity.badRequest().build();  
  
    }  
  
}
```

```
}  
  
// 2. Path sandbox  
  
Path filePath =  
m_files.resolveSafe(img.getName());  
  
  
  
// 3. File exists  
  
if (!Files.exists(filePath)) {  
  
    return ResponseEntity.notFound().build();  
  
}  
  
...  
  
}
```

ארבע הגנות עצמאיות - ImageName (Redundant Defenses): טיפוס שמוודא תקינות שם הקובץ ללא - extension whitelist. path separators - רק סיומות תמונה מותרות, resolveSafe() - path sandbox שמוודא שהקובץ נמצא בתיקיית uploads, ובדיקת קיום הקובץ. כל הגנה עצמאית – כך שגם אם אחת נכשלת, האחרות מגנות.

[image-ב]

controllers/ImageController.java

```
private static final Map<String, MediaType>
MEDIA_TYPES = Map.of( "jpg",
    MediaType.IMAGE_JPEG, "jpeg",
    MediaType.IMAGE_JPEG, "png",
    MediaType.IMAGE_PNG, "gif", MediaType.IMAGE_GIF,
    "webp", MediaType.parseMediaType("image/webp") );
...
```

```
byte[] data = Files.readAllBytes(filePath);
MediaType mediaType =
    MEDIA_TYPES.getOrDefault(ext,
    MediaType.APPLICATION_OCTET_STREAM); return
    ResponseEntity.ok().contentType(mediaType).body
    (data);
```

ה Content-Type - נקבע לפי map קבוע לפי הסיומת - לא לפי מה ששלח הדפדפן, ולא לפי Files.probeContentType. כך מובטח שהדפדפן יפרסר את הקובץ כתמונה בלבד ולא כ- HTML או JavaScript.

[image-ג]

controllers/ImageController.java

```
@RestController
public class ImageController{
    @GetMapping("/image")
    public ResponseEntity<byte[]> getImage(@RequestParam("img") ImageName
    img, Authentication auth){ ... }
}
```

קונטרולר נפרד: הגשת תמונות מופרדת לקונטרולר ייעודי – הפרדת אחריות ומניעת גישה לכל נתיב שירותי.

[image-ד]

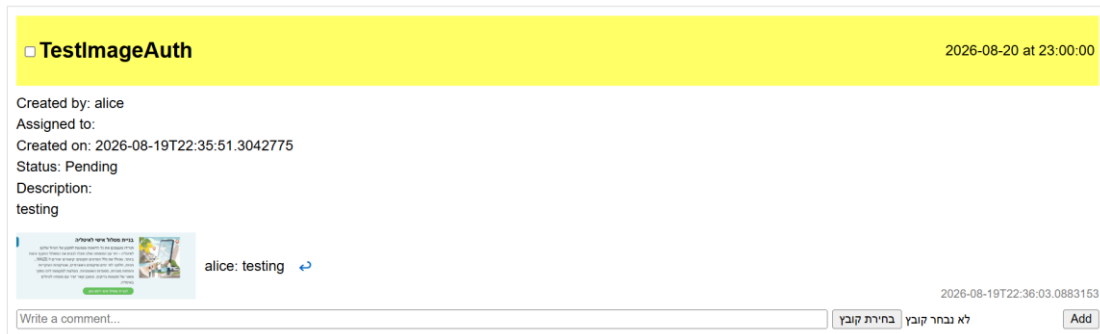
controllers/ImageController.java

```
@PreAuthorize("@taskAuth.imgIsInOwnedOrAssignedTask(#img, authentication)")
public ResponseEntity<byte[]> getImage(@RequestParam("img") ImageName img,
    Authentication auth)
```

הרשאות: משתמש יכול לגשת רק לתמונות השייכות ל tasks- שהוא בעליהן או שהוקצה אליהן – מונע גישה לתמונות של משתמשים אחרים.

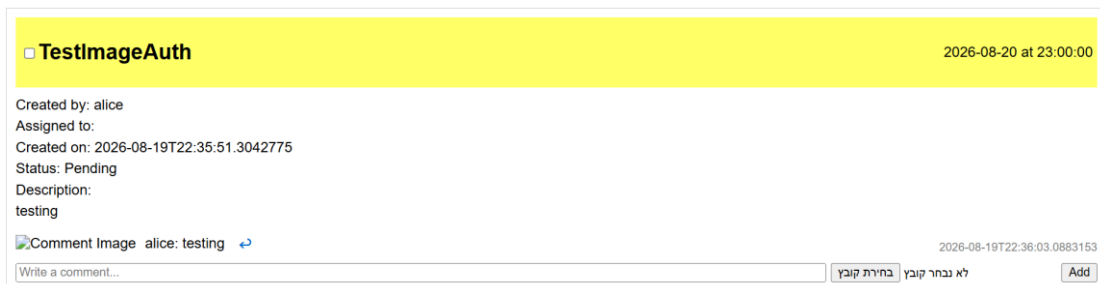
הוכחת הגנת הרשאות בזמן ריצה

צילום מסך 1 – alice מחוברת ורואה את התמונה שהיא עצמה העלתה למשימה שיצרה:

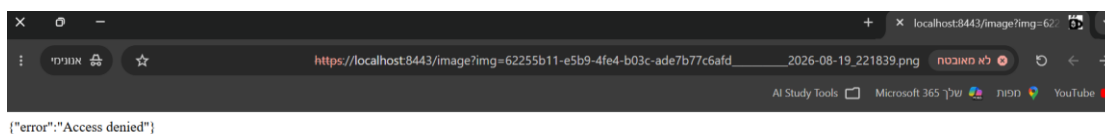


alice היא הבעלת המשימה, (createdBy: alice) ולכן הגישה לתמונה מותרת.

צילום מסך 2 bob מחובר, רואה את המשימה – התמונה מוצגת כ "Comment Image" - עבור bob - יכול לראות שקיימת תמונה, אבל הדפדפן לא מצליח לטעון אותה כי השרת חסם את הבקשה.



צילום מסך 3 גישה ישירה ל URL {"error": "Access denied"} - ניסיון גישה ישיר ל URL של התמונה מוחזר כשגיאת הרשאות.



HTTPS – חיבור מאובטח

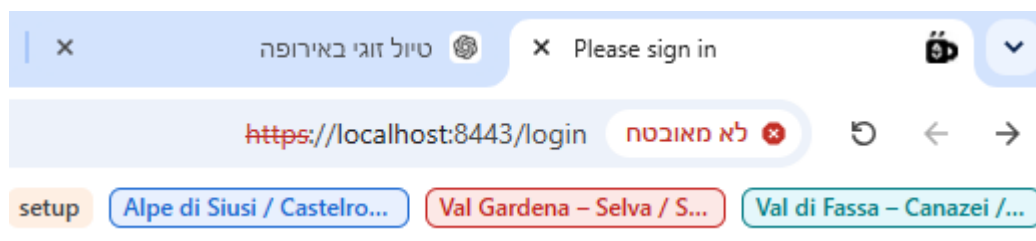
ההגדרות ב: src/main/resources/application.properties

```
server.port=8443
server.ssl.enabled=true
server.ssl.key-store=classpath:busybee.p12
server.ssl.key-store-type=PKCS12
server.ssl.key-alias=busybee
# Password read from the OS environment variable – NOT stored in this file
server.ssl.key-store-password=${KEYSTORE_PASSWORD}
server.ssl.enabled-protocols=TLSv1.3,TLSv1.2
```

השרת מאזין על פורט 8443 עם HTTPS בלבד. הקייסטור מסוג PKCS12 מכיל את המפתח הפרטי והאישור. סיסמת הקייסטור נקראת ממשתנה סביבה KEYSTORE_PASSWORD ולא מקודדת בקוד. רק TLS 1.2 ו-1.3 מותרים.

```
server.servlet.session.cookie.http-only=true
server.servlet.session.cookie.secure=true
server.servlet.session.cookie.same-site=Strict
server.servlet.session.cookie.name=BUSYBEE_SESSION
```

עוגיית הסשן מוגדרת כ-HttpOnly - בלתי נגישה ל JavaScript - מגן מפני XSS
Secure - נשלחת רק על HTTPS, ו SameSite = Strict מגן מפני CSRF.



האתר פועל על HTTPS פורט 8443. האזהרה מופיעה כי האישור הוא-self signed - לא מ CA מוכר, אך ההצפנה פעילה.

×

localhost: האישורים

כלליפרטים

מונפק ל

localhost

שם נפוץ (CN)

<לא חלק מהאישור>

ארגון (O)

<לא חלק מהאישור>

יחידה ארגונית (OU)

הונפק על ידי

localhost

שם נפוץ (CN)

<לא חלק מהאישור>

ארגון (O)

<לא חלק מהאישור>

יחידה ארגונית (OU)

תקופת תוקף

יום שלישי, 18 באוגוסט 2026 בשעה 17:39:25

יום רביעי, 18 באוגוסט 2027 בשעה 17:39:25

הונפק בתאריך

בתוקף עד

טביעות אצבע דיגיטליות מסוג SHA-256

אישור

70bd80b7ad1317475e0256937d66c2cad989bc8ebdd27bf4e1d9a2a979b6ac18

מפתח ציבורי

589bb537da42ef451081105cff64618bcd36e32d89d37453d6213420d9a8665

פרטי האישור: הונפק ל localhost - תוקף שנה, הצפנה SHA-256 .

הגנה מפני דליפת מפתחות ל Git:

.gitignore

```
### Security – never commit secrets or keystores ###
```

```
*.p12
```

```
*.jks
```

```
*.pfx
```

```
.env
```

```
*.env
```

```
launch.json
```

הסבר:

קובץ הgitignore מונע העלאה בטעות של קבצים רגישים למאגר הציבורי:

- *.p12 מונע העלאת קובץ ה Keystore-שמכיל את המפתח הפרטי. חשיפה תאפשר לתוקף לזייף את זהות השרת.
- launch.json מונע העלאת קובץ ה IDE-שמכיל את KEYSTORE_PASSWORD .
- .vscode/ מונע חשיפת הגדרות סביבה מקומיות.

מניעת Mixed Content

```
// Before lead to Mixed Content:  
  
// const baseUrl = "http://localhost:8080";  
  
  
// After (fixed):  
  
const baseUrl = "";  
  
async function sendPost(path, jsonOrFormData) {  
  const response = await fetch(`${baseUrl}${path}`, ...);  
}
```

כשהדף נטען דרך HTTPS, קריאות fetch לכתובת http://localhost:8080 היו נחסמות על ידי הדפדפן כ Mixed Content - הפתרון: שינוי ה baseUrl - לנתיב יחסי ריק. כך הדפדפן מסיק אוטומטית את הפרוטוקול (HTTPS) ואת הפורט (8443) מהדף הנוכחי - כל בקשת fetch פועלת על HTTPS בלבד.

Logging

הגדרת רמות הלוגים: application.properties

```
logging.level.org.springframework.security=WARN
```

```
logging.level.org.springframework.web=WARN
```

```
logging.level.com.securefromscratch=INFO
```

הסבר: לוגי Spring המובנים מוגדרים ל WARN-בלבד (כדי לא להציף), בעוד שקוד האפליקציה עצמה מוגדר ל INFO כך כל אירוע עסקי מתועד.

שורות Logging בקוד: TasksController.java

```
LOGGER.info("event=task_created id={} owner={} name={}", id, username, name);
```

```
LOGGER.info("event=task_done id={} by={}", taskid, authentication.getName());
```

```
LOGGER.warn("event=task_duplicate_name name={}", name);
```

CommentsUploadController.java:

```
LOGGER.info("event=comment_added taskid={} by={} hasFile={}", ...);
```

```
LOGGER.warn("event=comment_upload_rejected taskid={} reason={}", ...);
```

ImagesController.java:

```
LOGGER.info("event=image_requested img={} by={}", img.getName(),  
auth.getName());
```

```
LOGGER.warn("event=image_ext_rejected img={} ext={}", img.getName(), ext);
```

```
LOGGER.warn("event=image_not_found img={}", img.getName());
```

הסבר: כל פעולה רגישה מתועדת עם רמת INFO, וכל דחייה או שגיאה מתועדת עם WARN. הפורמט event=... מאפשר חיפוש וסינון קל בלוגים.

```

2026-08-23T20:37:21.566+03:00 INFO 33748 --- [BusyBee] [nio-8443-exec-4] c.s.busybee.controllers.TasksController : event=task_created id=82a7d7f9-7e69-4ed9-bab5-b30c13187d56 owner=alice name=Try
2026-08-23T20:37:21.653+03:00 WARN 33748 --- [BusyBee] [nio-8443-exec-2] c.s.busybee.auth.TaskAuthorization : event=authorization_denied action=view_image img=Wikimedia-Corona_Lockdown_Tirupur_Tamil_Nadu.jpg user=alice
2026-08-23T20:37:21.653+03:00 WARN 33748 --- [BusyBee] [nio-8443-exec-2] c.s.b.c.GlobalExceptionHandler : event=access_denied user=alice endpoint=/image
2026-08-23T20:37:21.655+03:00 WARN 33748 --- [BusyBee] [io-8443-exec-10] c.s.busybee.auth.TaskAuthorization : event=authorization_denied action=view_image img=wikimedia_Fresh_vegetable_stall.jpg user=alice
2026-08-23T20:37:21.655+03:00 WARN 33748 --- [BusyBee] [io-8443-exec-10] c.s.b.c.GlobalExceptionHandler : event=access_denied user=alice endpoint=/image
2026-08-23T20:37:21.660+03:00 ERROR 33748 --- [BusyBee] [nio-8443-exec-3] c.s.b.c.GlobalExceptionHandler : event=unexpected_error endpoint=/image

```

ניתן לראות בזמן ריצה INFO: על יצירת משימה, ו WARN - על ניסיון גישה לתמונה שלא שייכת למשתמש (event=authorization_denied).

בנוס CSRF Protection

הבעיה: מתקפת (Cross-Site Request Forgery) CSRF - אתר זדוני גורם לדפדפן של המשתמש לשלוח בקשות לשרת BusyBee מבלי שהמשתמש יודע, תוך שימוש ב-session cookie שנשמר בדפדפן.

הפתרון: Cookie-based CSRF Token.

קוד SecurityConfig.java:

```

CsrfTokenRequestAttributeHandler requestHandler = new
CsrfTokenRequestAttributeHandler();

requestHandler.setCsrfRequestAttributeName(null); // defer token loading

http.csrf(csrf -> csrf

    .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())

    .csrfTokenRequestHandler(requestHandler)

);

```

הסבר: השרת שולח cookie בשם XSRF-TOKEN שה-JavaScript יכול לקרוא (withHttpOnlyFalse). אתר זדוני לא יכול לקרוא את ה-cookie בגלל same-origin policy.

קוד helpers.js:

```
function getCookie(name) {  
    const value = `; ${document.cookie}`;  
    const parts = value.split(`; ${name}=`);  
    if (parts.length === 2) return parts.pop().split(';').shift();  
}  
  
async function sendPost(path, jsonOrFormData) {  
    const csrfToken = getCookie("XSRF-TOKEN");  
    const headers = {  
        "X-XSRF-TOKEN": csrfToken  
    };  
    // ...  
}
```

הסבר: ה JS - קורא את ה cookie - ומוסיף אותו כ header X-XSRF-TOKEN -
בכל בקשת POST . השרת מאמת שה header-תואם את ה cookie - בקשה
ממקור זר לא יכולה לזייף את ה header.

Sources Network Performance Memory Application Security Lighthouse Recorder

Log Disable cache No throttling

40,000 ms 50,000 ms 60,000 ms 70,000 ms 80,000 ms 90,000 ms 100,000 ms 110,000 ms 120,000 ms 130,000 ms 140,000 ms 150,000 ms 160,000 ms 170,000 ms

X Headers Payload Preview Response Initiator Timing Cookies

X-Frame-Options DENY

X-Xss-Protection 0

▼ Request Headers ☐ Raw

Accept	*/*
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	he-IL,he;q=0.9,en-US;q=0.8,en;q=0.7
Connection	keep-alive
Content-Length	87
Content-Type	application/json
Cookie	BUSYBEE_SESSION=88E78A5DEC94556E6A512911637C3DD3; XSRF-TOKEN=728cea9b-15b3-4f12-b468-3c4e554e4d31
Host	localhost:8443
Origin	https://localhost:8443
Referer	https://localhost:8443/create/create.html
Sec-Ch-Ua	"Not=A?Brand";v="99", "Google Chrome";v="151", "Chromium";v="151"
Sec-Ch-Ua-Mobile	?0
Sec-Ch-Ua-Platform	"Windows"
Sec-Fetch-Dest	empty
Sec-Fetch-Mode	cors
Sec-Fetch-Site	same-origin
User-Agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
X-Xsrf-Token	728cea9b-15b3-4f12-b468-3c4e554e4d31

ניתן לראות ב: Request Headers-הערך של X-Xsrf-Token זהה לערך ה-
XSRF-TOKEN שב Cookie - השרת מאמת את ההתאמה. אתר זדוני לא יכול
לקרוא את ה cookie ולכן לא יכול לזייף את הheader.

הגנת login וregistration: config/SecurityConfig.java

```
http.formLogin(form -> form
    .loginProcessingUrl("/login")
    .defaultSuccessUrl("/main/main.html", true)
    .failureUrl("/index.html?error=true")
    .permitAll()
);
...
.csrf(csrf -> csrf
    .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
    .csrfTokenRequestHandler(requestHandler)
)
```

נתיב login/ אינו מוחרג ממנגנון CSRF - כל בקשת התחברות חייבת בטוקן CSRF תקף. בקשה ללא טוקן נחסמת אוטומטית ומחזירה שגיאת 403. דף ה-login המסופק על ידי Spring Security כולל את הטוקן אוטומטית, כך שהתחברות דרך הטופס החוקי עובדת כמצופה.

גם נתיב register/ מוגן - כל בקשת הרשמה עוברת דרך sendPost ב-helpers.js שמצרף את ה-X-XSRF-TOKEN header אוטומטית.